

Pràctica 6: *Constraint Logic Programming*

Lògica en la Informàtica

FIB

Antoni Lozano

Q1 2025–2026

Objectius

Aquesta pràctica té com a objectiu:

- Resoldre problemes de *constrained logic programming*.
- Familiaritzar-se amb la llibreria **clpfd** de Swi Prolog.

En un problema amb restriccions tenim

- Un conjunt de variables pertanyents a certs dominis

$$x_1 \in D_1$$

$$x_2 \in D_2$$

$$\vdots$$

$$x_n \in D_n$$

- Un conjunt de restriccions sobre les variables

Exemple 1: SAT

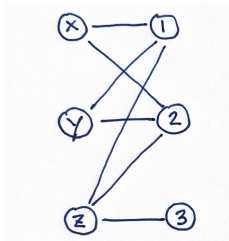
- Variables: $x_1, \dots, x_n$
- Domini: $\{0, 1\}$
- Restriccions: clàusules

Exemple 2: Sudoku

- Variables: $x_1, \dots, x_9$
- Domini: $\{1, \dots, 9\}$
- Restriccions:
 - files, columnes i blocks amb valors diferents
 - s'han d'importar els valors ja omplerts

Exemple 3: *Bipartite matching*

Volem trobar un *bipartite matching* en un graf com ara



- Variables: x, y, z
- Dominis: $x, y \in \{1, 2\}, z \in \{1, 2, 3\}$
- Restriccions: les variables han de ser diferents dos a dos

- En general, els problemes de programació amb restriccions són NP-complets.
- Cal entrellagar la generació i la comprovació i fer ús de la propagació (com en els *Sat solvers*)
- Farem servir la llibreria **clpfd** de Swi Prolog, declarant a l'inici

```
:- use_module(library(clpfd)).
```

Exemple 3: *Bipartite matching*

```
. :- use_module(library(clpfd)).  
.  
.  
main :-  
    X in 1..2,  
    Y in 1..2,  
    Z in 1..3,  
    all_different([X,Y,Z]),  
    label([X,Y,Z]),  
    write([X,Y,Z]).  
.
```

Coins

```
:- use_module(library(clpfd)).

exemple( 0, 26, [1,2,5,10]           ). % Solució: [1,0,1,2]
exemple( 1, 361, [1,2,5,13,17,35,157] ).

main :-
    exemple(0, Amount, Coins),
    nl, write('Paying amount '), write(Amount),
    write(' using the minimal number of coins of values '), write(Coins),
    nl, nl,

    %1: Variables i dominis:
    length(Coins, N),
    length(Vars, N), % obté una llista de N variables prolog
    Vars ins 0..Amount, % en el pitjor cas, només hi ha monedes de valor 1

    %2: Constraints:
    % per exemple expr( [X1,X2,X3,X4], [1,2,5,10], X4*10 + X3*5 + X2*2 + X1*1 )
    expr( Vars, Coins, Expr ),
    Expr #= Amount,

    %3: Labeling:
    % per exemple exprSuma( [X1,X2,X3,X4], X4+X3+X2+X1 )
    exprSuma( Vars, ExprSum ),
    labeling( [min(ExprSum)], Vars ),
```


Coins

```
.
%3: Labeling:
% per exemple exprSuma( [X1,X2,X3,X4], X4+X3+X2+X1 )
    exprSuma( Vars, ExprSum ),
    labeling( [min(ExprSum)], Vars ),

%4: Escrivim el resultat:
    NumCoins is ExprSum,
    write('We need '), write(NumCoins),
    write(' coins: '), write(Vars),
    nl, nl, halt.

% expr(+Vars, +Coins, ?Expr): atenció! això no calcula res!
% Només genera una estructura de dades prolog, que és una expressió
expr( [], [], 0 ).
expr( [X|Vars], [K|Coins], Expr + X*K ) :-
    expr( Vars, Coins, Expr ).

%exprSuma(+Vars, ?Expr)
exprSuma( [X], X ) :- !.
exprSuma( [X|Vars], X+Expr ) :-
    exprSuma( Vars, Expr ).
```

Tomografia

```
% A matrix which contains zeroes and ones gets "x-rayed" vertically and
% horizontally, giving the total number of ones in each row and column.
% The problem is to reconstruct the contents of the matrix from this
% information. Sample run:
%
%      ?- main.
%          0 0 7 1 6 3 4 5 2 7 0 0
%
%      0
%      0
%      8      * * * * * * *
%      2      *                      *
%      6      * * * * *
%      4      * *      * *
%      5      * *      * *
%      3      * *          *
%      7      * * * * *
%      0
%      0
%
%      :- use_module(library(clpfd)).

example1( [0,0,8,2,6,4,5,3,7,0,0], [0,0,7,1,6,3,4,5,2,7,0,0] ).
example2( [10,4,8,5,6], [5,3,4,0,5,0,5,2,2,0,1,5,1] ).
example3( [11,5,4], [3,2,3,1,1,1,1,2,3,2,1] ).
```

Tomografia

```
main :- example1(RowSums, ColSums),
%1: Variables i domini:
    length(RowSums, NumRows),
    length(ColSums, NumCols),
    NVars is NumRows*NumCols,
    length(L, NVars), % generate a list of Prolog vars (their names do not matter)
    ...
%2: Constraints:
    matrixByRows(L, NumCols, MatrixByRows),
    transpose(...
    declareConstraints(...
    ...
%3: Labeling:
    ...
%4: Escrivim el resultat:
    pretty_print(RowSums, ColSums, MatrixByRows).

pretty_print(_, ColSums, _) :- write('      '), member(S, ColSums), writef('%2r ', [S]), fail.
pretty_print(RowSums, _, M) :- nl, nth1(N, M, Row), nth1(N, RowSums, S), nl, writef('%3r ', [S]),
    member(B, Row), wbit(B), fail.
pretty_print(_, _, _)      :- nl.

wbit(1) :- write('* '), !.
wbit(0) :- write(' '), !.
```

Letter dice

```
:- use_module(library(clpfd)).

%% A (6-sided) "letter dice" has on each side a different letter.
%% Find four of them, with the 24 letters abcdefghijklmnoprstuvwxy such
%% that you can make all the following words: bake, onyx, echo, oval,
%% gird, smug, jump, torn, luck, viny, lush, wrap, fame.

% Some helpful predicates:

word( [b,a,k,e] ).
word( [o,n,y,x] ).
word( [e,c,h,o] ).
word( [o,v,a,l] ).
word( [g,i,r,d] ).
word( [s,m,u,g] ).
word( [j,u,m,p] ).
word( [t,o,r,n] ).
word( [l,u,c,k] ).
word( [v,i,n,y] ).
word( [l,u,s,h] ).
word( [w,r,a,p] ).
word( [f,a,m,e] ).
```

Letter dice

```
% num(?X, ?N)    "La lletra X és a la posició N de la llista"
num(X, N) :- nth1(N, [a,b,c,d,e,f,g,h,i,j,k,l,m,n,o,p,r,s,t,u,v,w,x,y], X ).

main :-
%1: Variables i dominis:
    length(D1, 6),
    length(D2, 6),
    length(D3, 6),
    length(D4, 6),
%2: Constraints:
    ...
%3: Labeling:
    ...
%4: Escrivim el resultat:
    writeN(D1), nl,
    writeN(D2), nl,
    writeN(D3), nl,
    writeN(D4), nl, halt.

writeN(D) :- findall(X, (member(N,D),num(X,N)), L), write(L), nl, !.
```